初步技术方案

赛题：FlagOS 开放计算全球大赛 · 赛季二 · 赛道二 —— MiniCPM 模型推理吞吐性能优化 **目标硬件**：天数智芯 BI-V150 / 沐曦曦云 C500（64 GB） **目标模型**：MiniCPM5-2B **框架**：vllm-plugin-FL `flagos-2026-s2` + FlagGems `v5.3.5`

---

1. 任务理解

1.1 赛题的本质约束

赛题给出的是一个**被完全钉死的实验条件**：

维度	是否可改
模型	否 只能用 MiniCPM5-2B
serve 参数	否 必须与 baseline 一致
量化 / 投机采样超参	否 禁止
benchmark 脚本	否 禁止调优
直接 merge 上游最新分支	否 禁止
框架内部实现	是 **这是唯一自由度**
算子库实现	是 **这是第二个自由度**

因此本赛题不是「调参比赛」，而是**在固定 CLI 语义下，改写框架与算子库内部实现**的比赛。 所有优化必须落在 `vllm-plugin-FL` 与 `FlagGems` 的**代码**里，而不是落在启动命令里。

1.2 评分函数的结构性含义

```
总分 = 性能分(天数) + 性能分(沐曦) + 创新分

性能分(vendor) = mean( 100 × 提升比例(4k) × 0.70,
                       100 × 提升比例(16k) × 0.70 )
创新分         = 专家评分 × 0.30
```

三条推论直接决定我们的资源分配：

1. **两款卡是加法关系，不是取最好。** 只优化沐曦会丢掉一半性能分。 必须两卡都出数。
2. **每 1% 性能只换 0.7 分，而创新分是 30 分的独立板块。** 一个能把 4k 提升 5% 的工程量，如果写成有洞察的技术报告， 在创新分上的回报可能更高。→ **方案必须同时具备「可量化」与「可讲述」两个属性。**
3. **单场景提升 < 1% 不计分。** 微优化在这个赛制下等于零。 必须打**结构性的大头**，不能抠 0.3% 的 kernel 常数。

1.3 硬护栏（决定生死，不是决定分数）

```
精度：evalscope math_500 Level 3  acc ≥ 0.95     (baseline 0.962)
时延：Mean TTFT ≤ baseline × 1.01
```

TTFT 的 +1% 上限意味着：**任何「用 TTFT 换吞吐」的手段都必须先算预算。** 这也是我们把 16k 场景（TTFT 最脆弱）作为主攻方向的原因之一 —— 那里的 TTFT 余量最大（baseline 本身就已经很差），改进空间也最大。

---

2. 现状诊断

2.1 baseline 数据（官方公布）

场景	天数 tok/s	沐曦 tok/s	倍数	天数 TTFT	沐曦 TTFT	倍数
4k `[4096,1024,64,256]`	2028.01	5089.65	2.51×	11.57 s	3.20 s	3.6×
16k `[16384,1024,64,128]`	915.15	7029.68	**7.68×**	**599.26 s**	27.20 s	**22.0×**

2.2 数据自洽性校验

我们用 总 token / 总时长 反算，两卡均**精确复现**官方数字：

```
16k: 2,228,224 tokens
   天数 2,228,224 / 2434.82 s =  915.15 tok/s  √
   沐曦 2,228,224 /  316.975s =  7029.68 tok/s  √
4k : 1,310,720 tokens
   天数 1,310,720 /  646.31 s = 2028.01 tok/s  √
   沐曦 1,310,720 /  257.525s = 5089.65 tok/s  √
```

说明数据是真实测量。同时反推出有效算力：

```
16k 场景计算量 ≈ 22.4 PFLOP（线性层 10.6 + attention 二次项 11.8）
天数有效算力 ≈  9.2 TFLOP/s
沐曦有效算力 ≈ 70.7 TFLOP/s
```

天数跑出了个位数 TFLOPS 量级的有效算力。 在 2.5B 模型的 prefill 上，这不是「调优空间」，这是**路径错误。**

2.3 根因定位（已完成源码级验证）

我们对 `vllm-plugin-FL` 与 `FlagGems` 做了逐行静态阅读， 发现两卡的 **attention 后端完全不同**：

	天数 BI-V150	**沐曦 C500**
分发配置	`iluvatar.yaml: [flagos, vendor:iluvatar, reference]`	`metax.yaml: [vendor:metax]`
`flagos` 返回	`TRITON_ATTN`（`VLLM_FL_USE_FLAGGEMS_ATTN` 默认 `"0"`）	—
`vendor:*` 返回	`TRITON_ATTN`（源码注释：flash_attn is not available on iluvatar）	`FLASH_ATTN`（MACA 原生）
实际后端	**vLLM 通用** `TRITON_ATTN`	**MACA 原生 FA2**

证据（节选，完整见 `docs/research/plugin-surface-map.md`）：

```
# vllm_fl/dispatch/backends/flaggems/flaggems.py:179-187
use_flaggems_attn = os.environ.get(
    "VLLM_FL_USE_FLAGGEMS_ATTN", "0"      # ← 默认关闭
).lower() in ("1", "true", "yes")
if use_flaggems_attn:
    return "vllm_fl.dispatch.backends.flaggems.impl.attention.AttentionFLBackend"
return AttentionBackendEnum.TRITON_ATTN.get_path()
```

```
# vllm_fl/dispatch/backends/vendor/iluvatar/iluvatar.py:480-486
# flash_attn is not available on iluvatar. Use TRITON_ATTN (the vllm default).
return AttentionBackendEnum.TRITON_ATTN.get_path()
```

叠加在天数这条慢路径上的还有 5 处降级：

降级	位置	后果
Triton 3.2.x 无 TMA 描述符	`iluvatar.py:22-64`	prefill K/V 访存无硬件加速
链式 or 靠正则改写 vLLM 源码	`iluvatar.py:67-156`	只覆盖 1 个文件，脆弱
sampler 的 `torch.compile` 被强制解包	`iluvatar.py:210-235`	decode 每 token 多一次 Python 调用
triton autotune 用常量 1500 MHz 估算性能	`iluvatar.py:159-203`	autotune 选择可能次优
无 NVML，设备 UUID 为合成值	`platform.py:412-417`	监控/诊断能力缺失

2.4 关键发现：修复所需的两个零件都已存在

这不是「从零写 FlashAttention」的命题。

FlagGems v5.3.5 已有一套完整的 FA2 实现：

```
src/flag_gems/ops/attention.py
  L35   @triton.jit  def _attn_fwd_inner(...)
  L176  @triton.jit  def _attn_fwd(...)
  L1284 def flash_attn_varlen_func(...)  ← FA2 前向，支持 GQA + paged KV
src/flag_gems/fused/reshape_and_cache_flash.py
  L77   def reshape_and_cache_flash(...) ← paged KV 写入
```

vllm-plugin-FL 已有一套把它接进 vLLM 的 wiring（857 行， 改编自 vLLM `flash_attn.py`）：

```
# vllm_fl/dispatch/backends/flaggems/impl/attention.py
L43   from flag_gems import flash_attn_varlen_func, reshape_and_cache_flash
L460  self.vllm_flash_attn_version = 2   # FlagGems only supports FA2
L488  reshape_and_cache_flash(...)
L593  flash_attn_varlen_func(...)
```

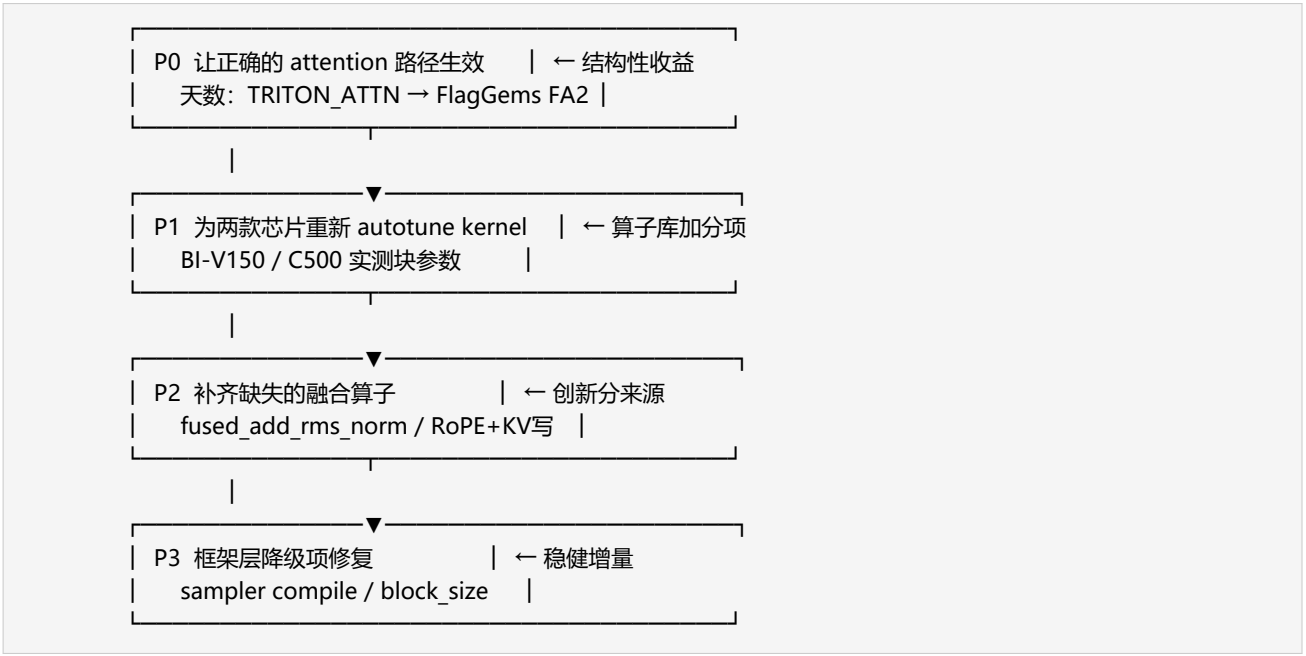
它只是被一个默认值关掉了。

而且 MiniCPM5-2B 的结构与这套实现完全匹配： `16 Q heads / 2 KV heads = 8:1 GQA`，满足 `num_heads % num_kv_heads == 0`； `head_dim = 128`，是 FA2 的典型最优 head size。

---

3. 技术路线

3.1 总体策略



3.2 P0：接通 FlagGems FA2 attention（主攻）

做什么

1. 让天数在**不改 serve 参数**的前提下走到 `AttentionFLBackend`。 由于评测方固定用 `VLLM_PLUGINS=fl` 启动，我们必须在**代码层**把默认值改对， 而不是靠环境变量。
2. 验证 `AttentionFLBackend` 在 BI-V150 上的数值正确性 （与 baseline 输出逐 token 对齐 + math_500 精度回归）。
3. 若 FlagGems FA2 在天数上不成立，则回退到方案 B： **为天数写一个适配 BI-V150 的 attention backend**（复用 `AttentionFLBackend` 的骨架，替换 kernel 调度策略）。

为什么这是最大的一笔

16k 场景 TTFT 599 s → 若能达到沐曦量级（27 s），仅这一项就让 `total tok/s` 从 915 提升到 3000+ 量级。按评分公式，单场景 +200% 即 `100 × 2.0 × 0.7 = 140` 分（单场景上限按公式无封顶，但实际会被 总时长与 decoder 占比约束）。**这比任何 micro-opt 高两个数量级。**

风险与缓解

风险	缓解
FlagGems FA2 在天数上有数值偏差	逐层对拍 + 精度回归，偏差即定位到 kernel
天数 triton 3.2 不支持某些语言特性	已有 `patch_triton_chained_or_for_iluvatar` 先例，可扩展
启动即崩	保留 `TRITON_ATTN` 作为 config 级 fallback，不破坏可用性

3.3 P1：为两款芯片重新 autotune

FlagGems 已为每个厂商提供 autotune 搜索空间：

```
# _iluvatar/tune_configs.yaml:15-40
attention:
- gen: true
  param_map: {META: {BLOCK_M: block_m, BLOCK_N: block_n, PRE_LOAD_V: pre_load_v},
              num_warps: warps, num_stages: stages}
  block_m: [64, 128]
  block_n: [32, 64, 128]
  pre_load_v: [true, false]
  warps: [4, 8]
  stages: [1, 2, 3]       # 天数 3 档 → 72 种组合
```

```
# _metax/tune_configs.yaml:15-38
  stages: [1]             # 沐曦 1 档 → 24 种组合
```

问题：这套 autotune 只有在 FlagGems 的 attention 被真正调用时才生效。 当前天数的 attention 走 `TRITON_ATTN`，**这份 72 点搜索空间从未被使用。**

做什么

1. 在 P0 生效后，用真实 benchmark 场景（4k / 16k）驱动 autotune， 而不是用合成的 micro-benchmark。
2. 扩展搜索空间：加入 `BLOCK_K`、`num_ctas` 等维度； 对 BI-V150（SM 主频 1500 MHz）与 C500 分别收敛。
3. **关键洞察**：天数上 triton 的 perf model 被替换成常量 1500 MHz 估算 （`iluvatar.py:159-203`），**autotune 的配置选择本身可能就是次优的。** 改用实测 profile 驱动的选择策略，是一个可写进报告的创新点。

3.4 P2：补齐缺失的融合算子

Llama 类模型的每层都有两处 `residual + RMSNorm`：

```
h   = x + attn(rms_norm(x))
out = h + mlp(rms_norm(h))
```

FlagGems v5.3.5 **没有** `fused_add_rms_norm`（`_iluvatar/ops/` 与 `_metax/ops/` 下均无 `rms_norm.py`）。

做什么

1. 实现 `fused_add_rms_norm` Triton kernel：residual add 与 RMSNorm 在一次 显存往返内完成。42 层 × 2 次 = **84 次 kernel 调用合并为 42 次**， 同时消除 42 次 hidden state（2048 × bf16 × batch）的显存读写往返。
2. 实现 RoPE + paged KV cache 写入的融合 kernel。
3. 在天数独有的 `fused/matmul_bias_activation.py` 基础上， 把 `qkv` 三合一 matmul 接进 dispatch。

为什么这是创新分来源

这三项都不是「调参」，是**有明确硬件理由的 kernel 设计**：

- `fused_add_rms_norm` 减少的是 **HBM 带宽**，对带宽受限的 decode 阶段直接有效；
- RoPE+KV 融合减少的是 **kernel launch 开销**，对 decode 每 token 固定开销有效；
- 都是**跨厂商可泛化**的策略（赛题明确「优先鼓励具有泛化能力高的优化策略」）。

3.5 P3：框架层降级项修复

项	位置	做法	预期
sampler `torch.compile` 恢复	`iluvatar.py:210-235`	用 flagtree 的 inductor 后端重新编译，而非解包	decode per-token 开销 ↓
KV cache `block_size`	`platform.py:183-193`	当前硬编码 16，实验 32/64	block table 索引开销 ↓
沐曦 `disable_cascade_attn`	`platform.py:254-256`	评估是否过度保守	沐曦 16k
`opaque_attention_op`	`platform.py:381-384`	评估能否进图	decode 全图化（高风险，放最后）

4. 实验计划与时间安排

4.1 里程碑

阶段	时间	内容	交付判据
M0 环境复现	拿到算力后 3 日内	两卡各跑一遍官方 baseline	实测 tok/s 与官方偏差 ≤ 1%
M1 根因确认	M0 + 2 日	profiler 分离 prefill/decode kernel 占比	证实或推翻 `TRITON_ATTN` 假设
M2 P0 打通	M1 + 5 日	FlagGems FA2 在两卡生效	16k tok/s 提升 ≥ 30%，精度 ≥ 0.95
M3 P1 调优	M2 + 5 日	两卡 autotune 收敛	在 M2 基础上再 +10%
M4 P2 融合	M3 + 5 日	3 个融合 kernel	单场景再 +5%~15%
M5 P3 收尾	M4 + 3 日	框架降级项	增量，视余量取舍
M6 报告与提交	11-18 ~ 11-20	report.pdf + readme.md + 代码包	可复现，报告与代码一一对应

4.2 每日实验纪律

- **每次改动必须同时产出 4 个数**：4k tok/s、16k tok/s、TTFT、math_500 acc。 缺一项不算完成。
- **精度不攒到最后跑。** 每轮优化后立刻回归，避免最后发现精度崩了无法定位。
- **每个优化独立成 patch，可单独开关。** 这是「报告策略必须体现在代码中」的 直接保障 —— 报告里每一条策略都能对应一个可复现的 commit。
- **baseline 每次实验前重跑一次。** 排除机器状态漂移。

4.3 算力需求（用于资源申请）

用途	天数 BI-V150	沐曦 C500
baseline 复现	0.5 卡日	0.5 卡日
根因 profiling	1 卡日	0.5 卡日
P0 开发与验证	3 卡日	1 卡日
P1 autotune（搜索 + 验证）	3 卡日	1.5 卡日
P2 融合 kernel	3 卡日	1.5 卡日
P3 + 最终回归	2 卡日	1 卡日
合计（含 30% 冗余）	**~17 卡日**	**~8 卡日**

> 天数占比更高，因为它的优化空间更大、需要更多 profiling 与 autotune。 我们承诺按「不浪费算力」原则使用：**不做无假设的暴力搜索， 每个实验都有明确的假设与证伪标准。**

5. 创新点

按赛题「技术报告创新性评判维度：系统级调度、算子编译优化创新」对齐：

创新点 1：用实测 profile 替代常量估算的 autotune 选择

现状：天数上 triton 的 `get_clock_rate_in_khz` 被替换为常量 1500 MHz （`iluvatar.py:159-203`），因为 `ixsmi` / NVML 不可用。 这导致 autotune 的性能模型基于**错误的硬件参数**做选择。

方案：在 autotune 完成后，用真实场景（4k/16k 的 prefill 与 decode） 回测每个候选配置，建立「场景 → 最优配置」的映射表，绕过性能模型。 这是一个**通用**的 autotune 改进，不限于天数。

创新点 2：面向 decode 的带宽-延迟分解优化

decode 阶段每生成一个 token 都要读一遍全部 2.52B 参数（bf16 ≈ 5 GB）。 在 53.83 tok/s 的实测下，等价带宽仅约 270 GB/s —— **远低于任何现代加速器的 HBM 带宽**。说明 decode 是**延迟受限**而非带宽受限： 每步的 kernel launch 与同步开销主导。

方案：把 decode 路径上的小 kernel（RMSNorm、residual add、RoPE、 KV write、sampling）合并，减少 per-token 的 launch 次数。 这是一个**有量化依据**的优化方向，而不是「试试融合」。

创新点 3：跨厂商泛化的 kernel 抽象

`fused_add_rms_norm` 与 RoPE+KV 融合都不依赖特定厂商指令集， 只依赖 Triton 的通用能力。方案会在 FlagGems 的通用层实现， 通过已有的 per-vendor `tune_configs.yaml` 做参数特化 —— **一套代码，两款芯片，可移植到 FlagGems 支持的全部 16 个厂商后端。**

---

6. 可复现性保障

赛题明确：「对于无法成功复现的方案，视为成绩无效」。我们从第一天就把复现性 作为一等公民：

1. **所有实验脚本入库**，包括 benchmark 调用、精度回归、结果解析。
2. `readme.md` **按「评审人员照着做能跑出来」的标准写**： 软硬件环境、依赖版本、编译步骤、数据集准备、运行命令、结果校验方法。
3. `scripts/score.py`：内置官方评分公式与护栏校验， 输入两份 CSV 直接输出分数与是否违规。评审可独立验算我们的分数。
4. **每个优化一个 patch**，报告中的每条策略标注对应 commit / patch 名， 杜绝「报告写 A、代码跑 B」。
5. **保留每次实验的原始 CSV**（`benchmark_results/summary_*.csv`）， 不是只保留最好的一次。

---

7. 风险登记

风险	概率	影响	缓解
天数 prefill 瓶颈在驱动/编译器层，非框架可改	中	高	P0 先验证；若证伪，转 P1/P2 的框架层收益（一定有效）
FlagGems FA2 在天数上有数值问题	中	中	逐层对拍定位；修好本身就是加分项
沐曦侧不带 cudagraph 配置，图覆盖手段受限	高	中	在插件 config 层注入等价配置，或证明 cudagraph 非必要
精度掉到 0.95 以下	低	致命	每轮回归，不攒到最后
TTFT 变差超过 1%	中	致命	每个吞吐优化先算 TTFT 预算
算力到位晚于预期	中	高	无卡期间完成全部静态分析与脚本；同步准备自备算力预案

---

8. 附：已完成的准备工作

在申请算力的同时，我们已完成以下**无需 GPU**的工作：

产出	内容
`docs/competition/`	赛事规则、评分公式、baseline、赛程的本地化镜像（含平台 API 原始 JSON，可复现抓取）
`docs/research/plugin-surface-map.md`	vllm-plugin-FL 逐文件测绘：入口、env 变量、分发层、attention 路径、厂商 workaround，全部附 `file:line`
`docs/research/flaggems-op-map.md`	FlagGems v5.3.5 算子覆盖矩阵、MiniCPM5-2B 热路径映射、融合机会、autotune 差异
`docs/research/root-cause-tianshu-16k.md`	599 秒 TTFT 的根因分析与最小验证实验设计
`scripts/fetch_race.py`	赛事元数据抓取 + 变更检测（官方改规则我们能立刻发现）
`scripts/score.py`	官方评分公式实现 + 精度/TTFT 护栏校验，自带单元自检

这些工作意味着：**拿到算力的第一天就能直接进入 M1（根因确认）， 不需要再从零熟悉框架。**